```
      extends Command                                                    ❶

  def apply(): Behavior[Command] =
    Behaviors.setup { context =>
      val adapter: ActorRef[Worker.Response] =                           ❷
        context.messageAdapter(response => WorkerDoneAdapter(response)) ❷

      Behaviors.receiveMessage { message =>
        message match {
          case Delegate(texts) =>
            forms.map { text =>
              val worker: ActorRef[Worker.Command] =
                context.spawn(Worker(), s"worker-$text")                 ❸
              context.log.info(s"sending task '$text to $worker")
              worker ! Worker.Parse(adapter, form)
            }
            Behaviors.same
          case WorkerDoneAdapter(Worker.Done(text)) =>                   ❹
            context.log.info(s"form '$form' has been finished")
            Behaviors.same
        }
      }
    }
}
```

❶ Adapter message

❷ Adapter function

❸ Spawns one worker per text

❹ Deals with the worker’s reply

This is everything: the manager’s protocol, the adapter message, and the delegate. The adapter function is used as an intermediary between the manager and the worker, and finally, the adapter message is used in the worker’s response. Next, let’s discuss the worker’s protocol.

## *3.1.7 A protocol with commands and responses as traits*